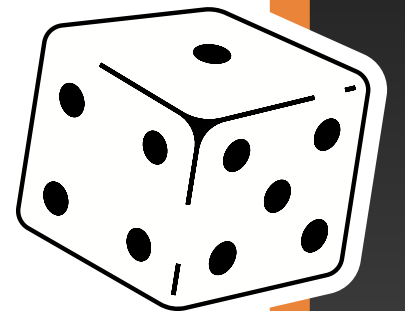




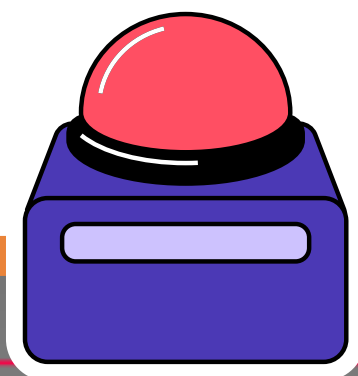
INHERITANCE, POLYMORPHISM, REFERENCING, SUPER

ALGORITHMS & OBJECT-ORIENTED PROGRAMMING

WEEK 11

JOSEPH ANDREAS

FOR INTERNAL USE ONLY



AGENDA

1. INHERITANCE

2. POLYMORPHISM

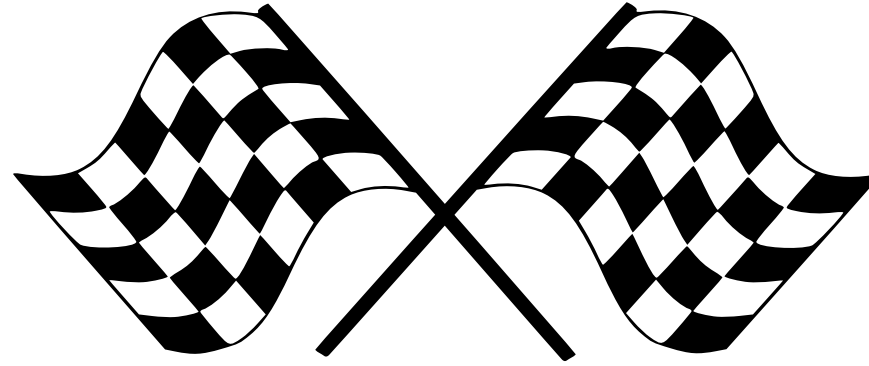
3. OBJECT REFERENCING

4. SUPER

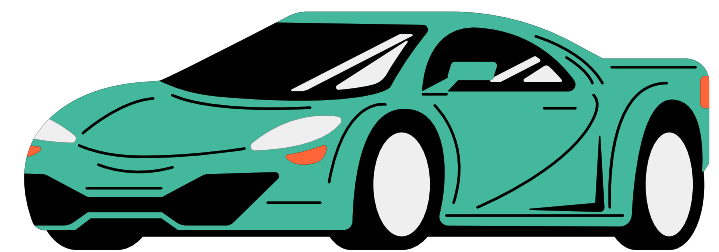
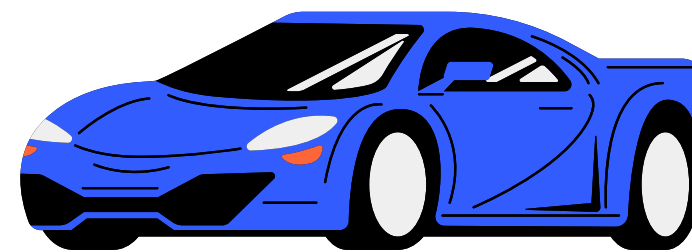
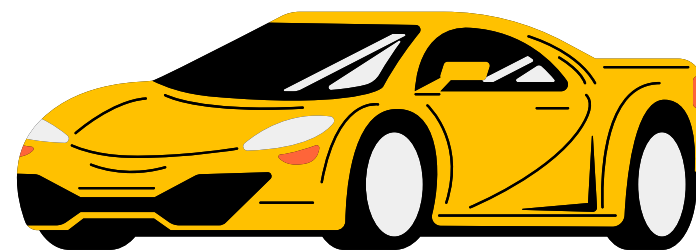
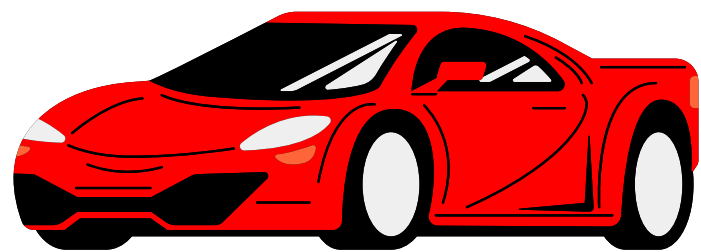


START





INHERITANCE



WHAT IS INHERITANCE ?

- Allows a class to inherit properties and behavior from an existing class
- Terminologies:
 - child class (subclass): Class which inherits properties and behavior from its parent class. Uses the `extend` keyword to inherit from parent class
 - parent class (superclass): Class that has its properties and behavior inherited to the child class

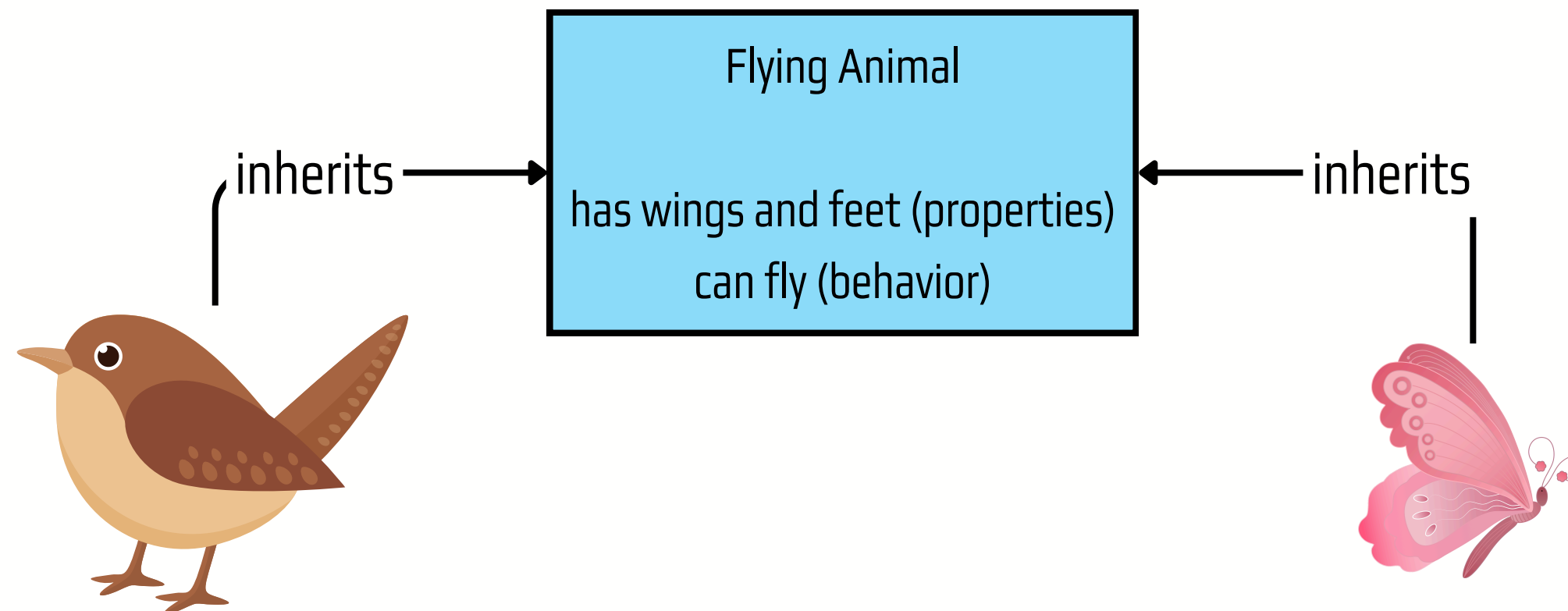
WHAT IS INHERITANCE ?

- To put into perspective:
 - A flying animal has wings and feet as its properties, and can fly as their behavior
 - A bird is a flying animal, therefore it can inherit properties of flying animals
 - A butterfly is also a flying animal, therefore it can also inherit properties of flying animals

A class can only inherit from one parent class (cannot inherit from multiple parent classes)

WHAT IS INHERITANCE ?

- To put into perspective:
 - A flying animal has wings and feet as its properties, and can fly as their behavior
 - A bird is a flying animal, therefore it can inherit properties of flying animals
 - A butterfly is also a flying animal, therefore it can also inherit properties of flying animals



EXAMPLE OF INHERITANCE

```
class FlyingAnimal {  
    String wings = "I have wings";  
    String feet = "I have feet";  
  
    void fly(){  
        System.out.println("Ich bin  
fliegen");  
    }  
}  
  
class Bird extends FlyingAnimal {  
}  
  
class Butterfly extends FlyingAnimal {  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Bird bird = new Bird();  
        bird.fly();  
        System.out.println("Bird says " +  
bird.wings + " and " + bird.feet);  
        Butterfly butterfly = new Butterfly();  
        butterfly.fly();  
        System.out.println("Butterfly says " +  
butterfly.wings + " and " + butterfly.feet);  
    }  
}
```

FlyingAnimal is parent class
(superclass).

Bird and **Butterfly** are child classes
(subclass), which inherits
characteristics of **FlyingAnimal** class

EXAMPLE OF INHERITANCE

```
class FlyingAnimal {
    String wings = "I have wings";
    String feet = "I have feet";

    void fly(){
        System.out.println("Ich bin fliegen");
    }
}

class Bird extends FlyingAnimal {
    String mouth = "beak";
}

class Butterfly extends FlyingAnimal {
    String antennae = "antennae";
}
```

```
public class Main {
    public static void main(String[] args) {
        Bird bird = new Bird();
        bird.fly();
        System.out.println("Bird says " +
            bird.wings + " and " + bird.feet " and " +
            bird.mouth);
        Butterfly butterfly = new Butterfly();
        butterfly.fly();
        System.out.println("Butterfly says " +
            butterfly.wings + " and " + butterfly.feet +
            " and " + butterfly.antennae);
    }
}
```

FlyingAnimal is parent class
(superclass).

Bird and **Butterfly** are child classes
(subclass), which inherits
characteristics of **FlyingAnimal** class

Possible to add properties that are
unique to that particular child class

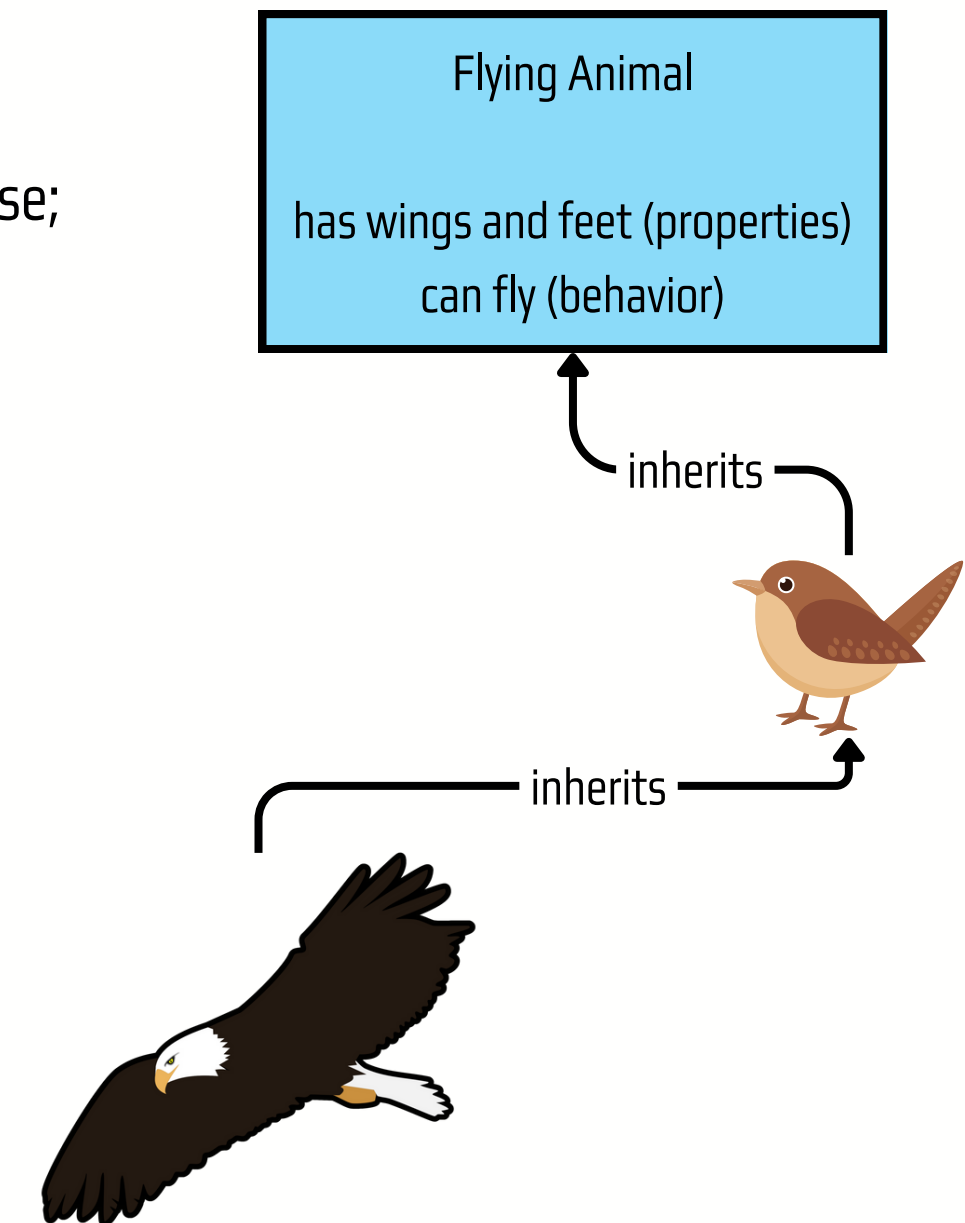
EXAMPLE OF INHERITANCE

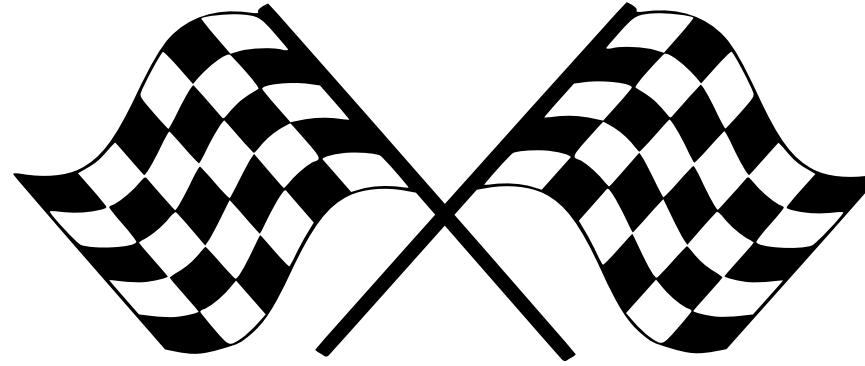
```
class FlyingAnimal {  
    String wings = "I have wings";  
    String feet = "I have feet";  
  
    void fly(){  
        System.out.println("Ich bin  
fliegen");  
    }  
}  
  
class Bird extends FlyingAnimal {  
    String mouth = "beak";  
}  
  
class Eagle extends Bird {  
    String food = "meat";  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Eagle eagle = new Eagle();  
        eagle.fly();  
        System.out.println("I am an eagle. " +  
eagle.wings + " " + eagle.feet + " and I eat "  
+ eagle.food + " with my " + eagle.mouth);  
    }  
}
```

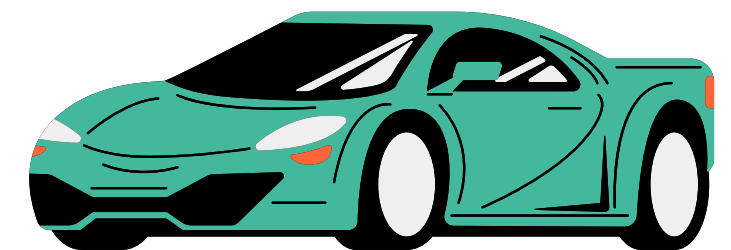
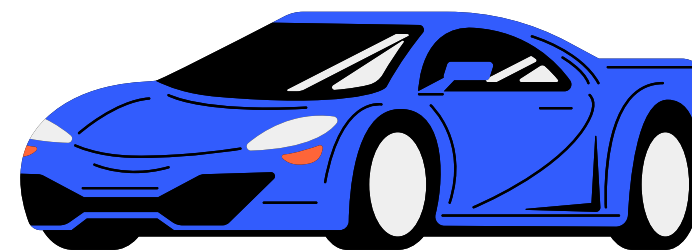
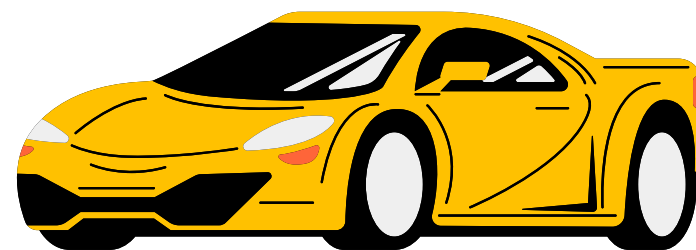
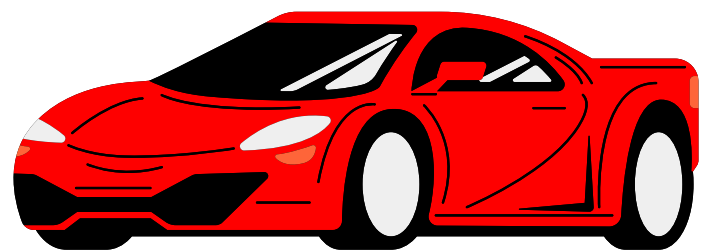
A child class can also be extended to another child class

In this case;





POLYMORPHISM



WHAT IS POLYMORPHISM ?

- Polymorphism:
 - Allows objects to be treated as instances of their parent class
 - Enables methods to take on different forms based on the object that calls them
 - Can be achieved through:
 - Method overloading (compile-time polymorphism): Occurs within the same class, define multiple methods with the same name but with different parameters. Resolved at compile time
 - Method overriding (run-time polymorphism): Occurs across different classes in an inheritance hierarchy. Method name, return type, and parameters must match with its parent class. Resolved at run time

POLYMORPHISM EXAMPLE

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes  
a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog myDog = new Dog();  
        Cat myCat = new Cat();  
  
        myDog.sound();  
        myCat.sound();  
    }  
}
```

Animal is the parent class

Dog is a form of **Animal** with a different characteristic

Cat is also a form of **Animal** with another different characteristic

METHOD OVERRIDING EXAMPLE

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes  
a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog myDog = new Dog();  
        Cat myCat = new Cat();  
  
        myDog.sound();  
        myCat.sound();  
    }  
}
```

Different Classes: Method overriding occurs across different classes in an inheritance hierarchy

Same Method: The method name, return type, and parameters must match

Run-time Resolution: The method to be called is determined at runtime based on the object type

METHOD OVERLOADING EXAMPLE

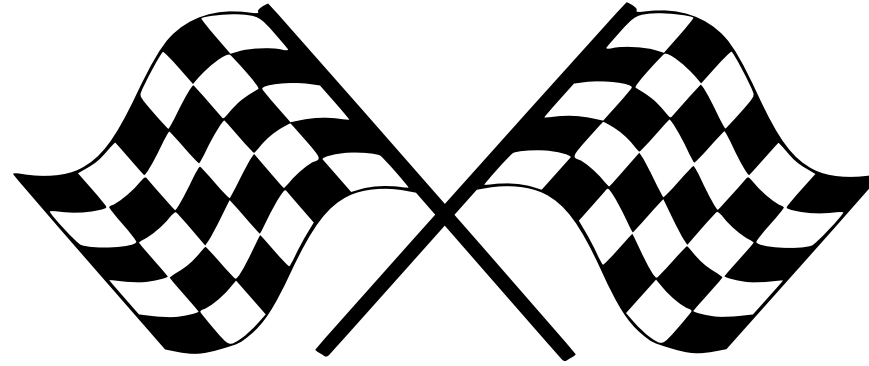
```
class MathOperations {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    String add(String a, String b){  
        String text = a + b;  
        return text;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MathOperations math = new  
        MathOperations();  
  
        System.out.println("Addition of two  
texts: " + math.add("Sleepy", "Hungry"));  
        System.out.println("Addition of three  
integers: " + math.add(5, 10, 15));  
        System.out.println("Addition of two  
doubles: " + math.add(5.5, 10.5));  
    }  
}
```

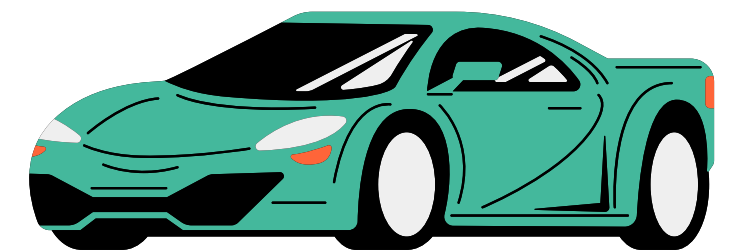
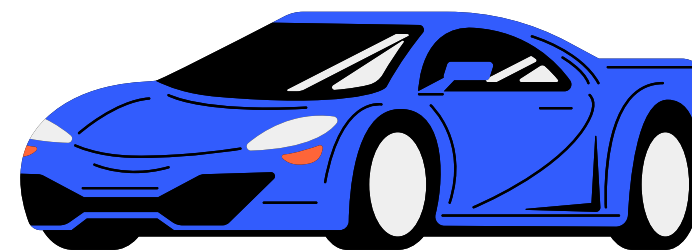
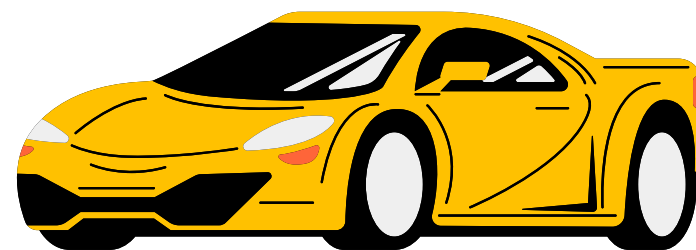
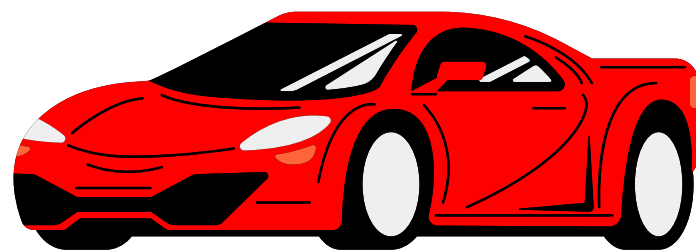
Same Class: Method overloading occurs within the same class

Different Parameters: Define multiple methods with the same name but with different parameter lists

Compile-time Resolution: The correct method is determined at compile time based on the arguments passed to the method



OBJECT REFERENCING



LET'S HAVE ANOTHER EXAMPLE

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
    void fetch() {  
        System.out.println("Dog fetches ball");  
    }  
}
```


WHAT IS AN OBJECT REFERENCE?

- Reference: A variable that points to an object stored in memory

```
Dog d = new Dog();
```

- new Bird() → creates a Dog object in memory
 - Bird b → creates a reference variable
 - b points to that Bird object
- A reference does NOT contain the object, it only points to it

REFERENCE TYPE VS OBJECT TYPE

- Reference Type = Declared type of the variable
- Object Type = Real object created with `new`

```
Dog dog = new Dog();
```

Reference type: Dog
Object type: Dog

```
Animal dog = new Dog();
```

Reference type: Animal
Object type: Dog

These two behave differently

WHY DOES THE REFERENCE TYPE MATTER?

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
  
    void fetch() {  
        System.out.println("Dog fetches ball");  
    }  
}
```

Reference type determines which methods are allowed to be called (not which version of the method runs)

```
Animal a = new Dog();  
a.fetch(); → Error because Animal does not have fetch()
```

WHY DOES THE OBJECT TYPE MATTER?

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
  
    void fetch() {  
        System.out.println("Dog fetches ball");  
    }  
}
```

Object type determines which overridden method actually runs at runtime

```
Animal a = new Dog();  
a.sound(); → Dog barks
```

- Allowed because `Animal` has `sound()`
- Runs `Dog's` version because the object is a `Dog`

This is called dynamic method dispatch

SO WHAT'S THE DIFFERENCE?

Code	Reference Type	Object Type	What methods can be called?
<code>Dog dog = new Dog();</code>	Dog	Dog	Dog + Animal methods
<code>Animal dog = new Dog();</code>	Animal	Dog	Only Animal methods (Dog method overrides still run)

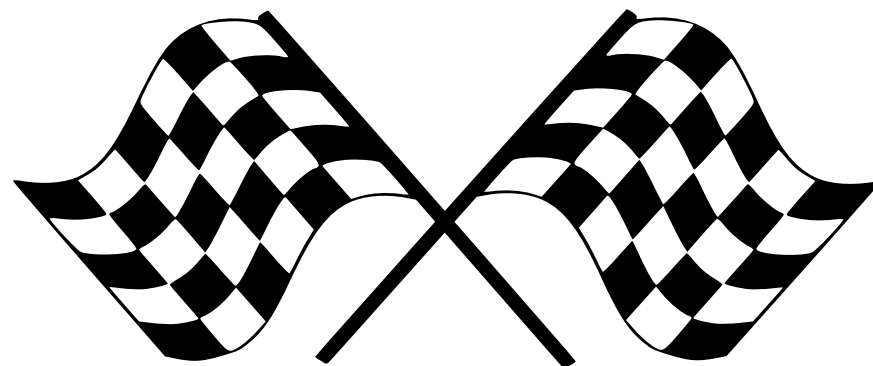
EXAMPLES

Code	When to use?		
<pre>Dog dog = new Dog(); dog.fetch(); dog.sound();</pre>	Need Dog-only methods	Working specifically with dogs	Want full access to everything Dog can do
<pre>Animal[] animals = { new Dog(), new Cat(), new Cow() }; for (Animal a : animals) { a.sound(); }</pre>	Only need Animal-level behavior	The code must work with multiple kinds of animals	Want a more flexible code which implements polymorphism

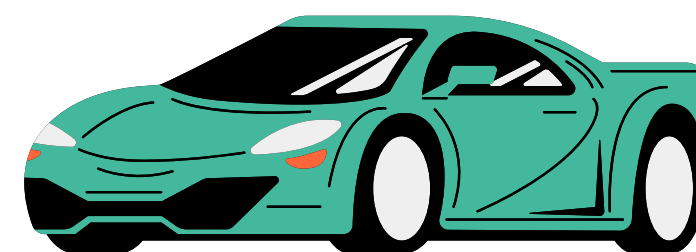
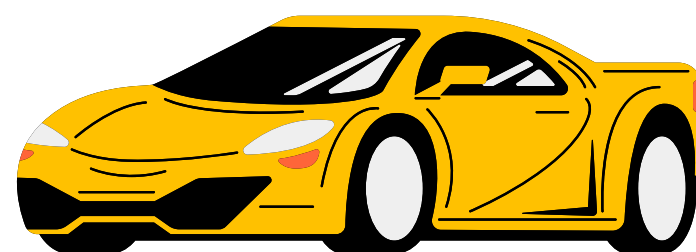
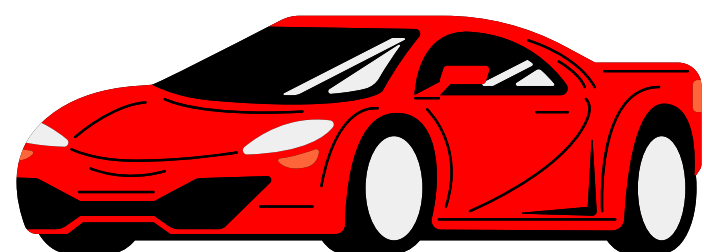
KEY TAKEAWAYS

Situation	Use Child Reference	Use Parent Reference
Need child-only methods	Yes	No
Don't need flexibility	Yes	No
Want polymorphism	No	Yes
Want to store many types together	No	Yes
Code depends on shared/common method	No	Yes

- Working with a specific type → use the specific type (Dog)
- Want flexible, reusable, polymorphic code → use the parent type (Animal)
- Need only the common methods → use the parent/interface type
- Need specific methods → use the child type



SUPER



SUPER !

- Super: A keyword that is used to refer to the members of the parent class / superclass (attributes, methods, and constructors) from a child class / subclass. Can be used to:
 - Call superclass constructor: `super()` or `super(parameters)`
 - Access superclass fields: `super.field`
 - Call superclass methods: `super.method()`

A SUPER EXAMPLE !

```
class Animal {  
    public void animalSound() {  
        System.out.println("The animal  
makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    public void dogSound() {  
        super.animalSound();  
        System.out.println("The dog says:  
bow wow");  
    }  
}
```

```
public class SuperDemo {  
    public static void main(String args[]) {  
        Dog myDog = new Dog();  
        myDog.dogSound();  
    }  
}
```

Animal is the parent class, and has the method `animalSound`

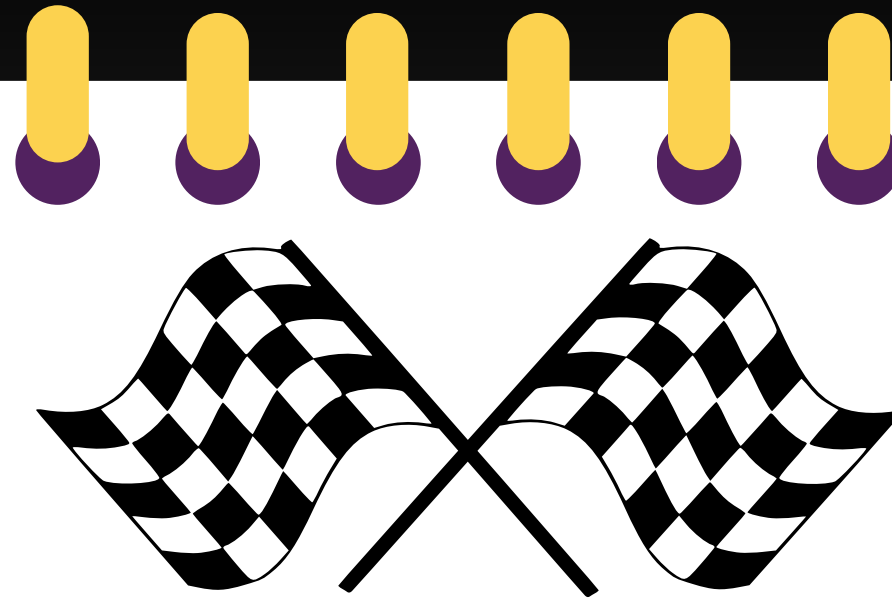
Dog is the child class, which has a method `dogSound` where it calls the `animalSound` method with the `super` keyword, as well as adding another behavior

ANOTHER SUPER EXAMPLE !

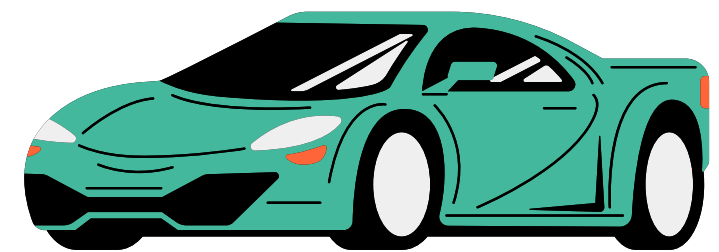
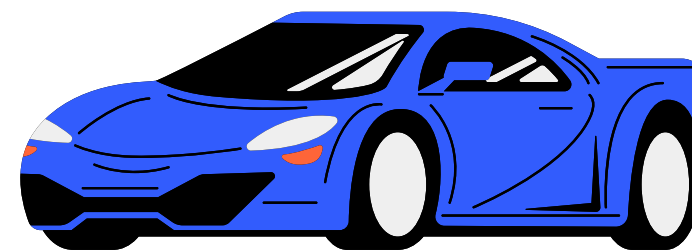
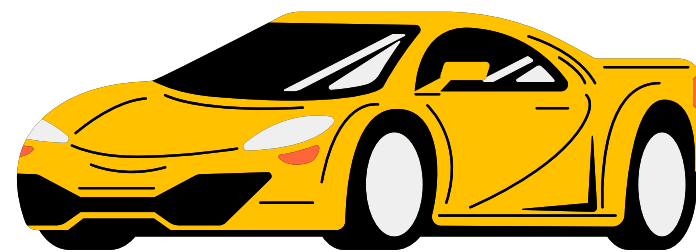
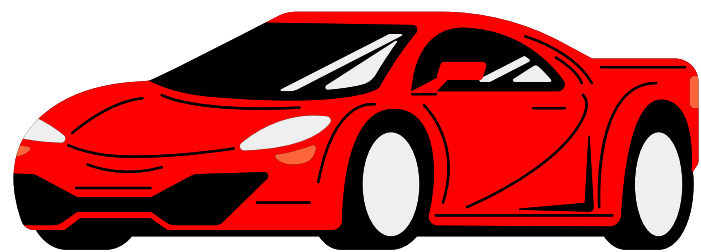
```
class Vehicle {  
    String brand;  
  
    Vehicle(String brand) {  
        this.brand = brand;  
    }  
  
    void honk() {  
        System.out.println("Beep beep!");  
    }  
}
```

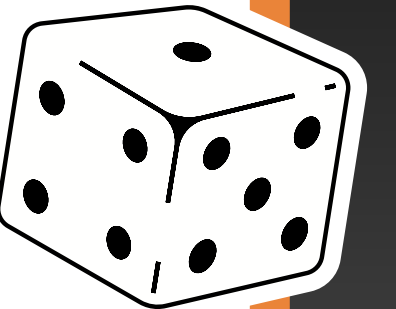
```
class Car extends Vehicle {  
    String modelName;  
  
    Car(String brand, String modelName) {  
        super(brand);  
        this.modelName = modelName;  
    }  
  
    void displayDetails() {  
        System.out.println("Brand: " +  
super.brand);  
        System.out.println("Model: " +  
this.modelName);  
    }  
}
```

```
public class SuperDemo2 {  
    public static void main(String[] args) {  
        Car myCar = new Car("Ford",  
"Mustang");  
        myCar.honk();  
        myCar.displayDetails();  
    }  
}
```



ANY QUESTIONS?





THANK YOU FOR
YOUR ATTENTION!

